

TECHNICAL DOCUMENTATION

Trao AI Travel Planner

Phase 2 — Database Connection & Schemas



PHASE	COMMIT	DATE	AUTHOR
02 / 12	feat: add MongoDB connection and models	June 2026	Abhi

Phase 2 sets up our data layer. We establish a robust connection to MongoDB via **Mongoose** and build the schema structures for **Users** (auth information) and **Trips** (itinerary structure, hotel options, estimated budget, and custom packing assistant list).

2 Codebase Implementation

1. Config: backend/config/db.js

backend/config/db.js

JavaScript

```
// Import the Mongoose library to interact with MongoDB
import mongoose from 'mongoose';

/**
 * Asynchronously connects to the MongoDB database using the connection string from environment
 * Under Node.js --env-file option, process.env will automatically contain the MONGO_URI.
 */
const connectDB = async () => {
  try {
    // Attempt to establish a connection to the MongoDB database
    const conn = await mongoose.connect(process.env.MONGO_URI);

    // Log a success message containing the connection host to confirm connectivity
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    // Log any errors that occur during the connection attempt
    console.error(`MongoDB Connection Error: ${error.message}`);

    // Exit the process with failure code (1) to prevent the server from running without a database
    process.exit(1);
  }
};

// Export the connectDB function as the default export of this module
export default connectDB;
```

2. Model: backend/models/User.js

backend/models/User.js

JavaScript

```
// Import the Mongoose library to define schemas and models
import mongoose from 'mongoose';

// Define the schema for the User model
const UserSchema = new mongoose.Schema(
  {
    // The email address of the user, used for registration and login authentication
    email: {
```

```

    // Data type is string
    type: String,
    // Field is mandatory; registration will fail if it's missing
    required: [true, 'Please provide an email address'],
    // Email must be unique in the database; prevents multiple accounts with the same email
    unique: true,
    // Auto-convert to lowercase to ensure email matches are case-insensitive
    lowercase: true,
    // Trim leading and trailing whitespace characters
    trim: true
  },
  // The hashed password of the user
  password: {
    // Data type is string
    type: String,
    // Field is mandatory; registration will fail if password is missing
    required: [true, 'Please provide a password']
  }
},
{
  // Automatically manage createdAt and updatedAt fields for each user document
  timestamps: true
}
);

// Create the Mongoose model named 'User' using the schema definition
const User = mongoose.model('User', UserSchema);

// Export the User model as the default export of this module
export default User;

```

3. Model: backend/models/Trip.js

backend/models/Trip.js

JavaScript

```

// Import the Mongoose library to define schemas and models
import mongoose from 'mongoose';

// Define the detailed activity schema used within the daily itinerary array
const ActivitySchema = new mongoose.Schema({
  // The title of the specific activity (e.g. "Visit Eiffel Tower")
  title: {
    type: String,
    required: [true, 'Activity title is required']
  },
  // The description details of what the activity entails

```

```

description: {
  type: String
},
// The estimated cost in USD for this activity
estimatedCostUSD: {
  type: Number,
  default: 0
},
// The time of the day the activity takes place
timeOfDay: {
  type: String,
  enum: ['Morning', 'Afternoon', 'Evening']
}
});

// Define the itinerary schema representing a single day in the travel plan
const DailyItinerarySchema = new mongoose.Schema({
  // The number of the day in the trip sequence (e.g. 1, 2, 3...)
  dayNumber: {
    type: Number,
    required: [true, 'Day number is required']
  },
  // Array of activities planned for this specific day
  activities: [ActivitySchema]
});

// Define the schema for hotel recommendations for the trip
const HotelSchema = new mongoose.Schema({
  // The name of the recommended hotel
  name: {
    type: String,
    required: [true, 'Hotel name is required']
  },
  // The tier or standard of the hotel (e.g. budget, premium, luxury)
  tier: {
    type: String
  },
  // The estimated cost per night in USD
  estimatedCostNightUSD: {
    type: Number
  },
  // The user rating or star rating of the hotel (e.g. "4.5 stars")
  rating: {
    type: String
  }
});

// Define the schema for tracking estimated expenses in different categories
const EstimatedBudgetSchema = new mongoose.Schema({

```

```

// Cost estimated for transportation (flights, local transport)
transport: {
  type: Number,
  default: 0
},
// Cost estimated for accommodation (hotels, stays)
accommodation: {
  type: Number,
  default: 0
},
// Cost estimated for dining/food
food: {
  type: Number,
  default: 0
},
// Cost estimated for planned activities
activities: {
  type: Number,
  default: 0
},
// Total summed budget calculated across all categories
total: {
  type: Number,
  default: 0
}
});

// Define the schema for a single packing list item
const PackingItemSchema = new mongoose.Schema({
  // The name/description of the item to pack
  item: {
    type: String,
    required: [true, 'Packing item name is required']
  },
  // Category of the item to pack
  category: {
    type: String,
    enum: ['Documents', 'Clothing', 'Gear', 'Other']
  },
  // Checked status indicating if the item has been packed by the user
  isPacked: {
    type: Boolean,
    default: false
  }
});

// Define the core Mongoose schema for the Trip model
const TripSchema = new mongoose.Schema(
  {

```

```

// The reference ID to the User model, showing which user owns this trip
userId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'User',
  required: true
},
// The destination of the travel plan (e.g., "Paris, France")
destination: {
  type: String,
  required: [true, 'Destination is required']
},
// The total duration of the trip in days
durationDays: {
  type: Number,
  required: [true, 'Duration in days is required']
},
// Budget level indicator
budgetTier: {
  type: String,
  enum: ['Low', 'Medium', 'High'],
  required: [true, 'Budget tier is required']
},
// User interest tags for customize itinerary matching (e.g. ['hiking', 'museums'])
interests: [
  {
    type: String
  }
],
// The day-by-day travel plan generated for the user
itinerary: [DailyItinerarySchema],
// List of recommended hotel options for the trip
hotels: [HotelSchema],
// Budget breakdown calculations
estimatedBudget: {
  type: EstimatedBudgetSchema,
  default: () => ({}),
},
// Complete checklist of items to pack, categorized and toggleable
packingList: [PackingItemSchema]
},
{
  // Automatically add and manage createdAt and updatedAt fields for auditing
  timestamps: true
}
);

// Create the Mongoose model named 'Trip' using the schema definition
const Trip = mongoose.model('Trip', TripSchema);

```

```
// Export the Trip model as the default export of this module
export default Trip;
```

3 Phase 2 Completion Checklist

- ✓ **backend/config/db.js** implemented using Mongoose connection and ES Module syntax
- ✓ **backend/models/User.js** created with required schema constraints (email lowercase, trimmed, unique)
- ✓ **backend/models/Trip.js** created matching all requested fields, types, default values, and subdocument arrays exactly
- ✓ Detailed inline comments added to every single line explaining its exact purpose

📦 GIT COMMIT FOR THIS PHASE

```
git commit -m "feat: add MongoDB connection and Mongoose User and Trip models"
```